```
        _userManager = userManager;                                    ❷
    }                                                                  ❷

    public async Task<IViewComponentResult> InvokeAsync(               ❸
        int numberOfRecipes)                                           ❹
    {
        if(!User.Identity.IsAuthenticated)
        {
            return View("Unauthenticated");                            ❺
        }

        var userId = _userManager.GetUserId(HttpContext.User);         ❻
        var recipes = await _recipeService.GetRecipesForUser(          ❻
            userId, numberOfRecipes);

        return View(recipes);                                          ❼
    }
}
```

❶ Deriving from the ViewComponent base class provides useful methods like View().

❷ You can use DI in a view component.

❸ InvokeAsync renders the view component. It should return a Task<IViewComponentResult>.

❹ You can pass parameters to the component from the view.

❺ Calling View() will render a partial view with the provided name.

❻ You can use async external services, allowing you to encapsulate logic in your business domain.

❼ You can pass a view model to the partial view. Default.cshtml is used by default.

This custom view component handles all the logic you need to render a list of recipes when the user is logged in or a different view if the user isn't authenticated. The name of the view component is derived from the class name, like Tag Helpers. Alternatively, you can apply the `[ViewComponent]` attribute to the class and set a different name entirely.

The `InvokeAsync` method must return a `Task<IViewComponentResult>.` This is similar to the way you can return `IActionResult` from an action method or a page handler, but it's more restrictive; view components must render some sort of content,